

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
FACULTAD DE INGENIERÍA
DIVISIÓN DE INGENIERÍA ELÉCTRICA
INGENIERÍA EN COMPUTACIÓN
LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE EJERCICIO EN CLASE Nº 10

NOMBRE COMPLETO: Gómez Aceves Abraham Agustín

Nº de Cuenta: 320093290

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 05

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 3 – Noviembre – 2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

3.- Conclusión:

a. Los ejercicios del reporte: Complejidad, Explicación.

b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica

c. Conclusión

4. Bibliografía en formato APA

Sección 1.

Descripción del ejercicio

Ejercicio de práctica 10: Animación por Keyframes. Se pide: (1) agregar la función de escribir/sobreescribir y leer un archivo externo llamado keyframescohete.txt; y (2) crear POR TECLADO los cuadros clave (keyframes) de la animación para que el cohete despegue subiendo 8 unidades en Y siguiendo una curva hacia X positivo (a la derecha), y al llegar a esas 8 unidades empiece a bajar en picada, llegue al suelo y se quede acostado. Como evidencia se entregan las capturas de cada keyframe creado por teclado, el video y el archivo txt.

• **Base del ejercicio.** Se partió del proyecto con el código base de keyframes de la Práctica 10 (KeyFrame[], saveFrame, resetElements, interpolation y animate). En lugar del helicóptero del código base, ahora los keyframes animan al cohete de la Práctica 3, hecho con instancias de primitivas (cilindro, cono, esfera, cubo y pirámide), que ya se había pasado al formato de MeshModel en la práctica 9-2. El cohete está sobre una plataforma en (8, 0.9, -28), parado sobre el piso.

• **Datos de cada keyframe e interpolación.** Cada keyframe guarda tres valores: movCohete_x y movCohete_y (desplazamiento respecto a la plataforma) y giroCohete (giro sobre Z, en grados). Al reproducir, interpolation() calcula para cada valor el incremento = (siguiente − actual) / i_max_steps, con i_max_steps = 90, y animate() suma ese incremento en cada cuadro, así que entre dos keyframes hay 90 cuadros (1.5 s a 60 fps). La matriz del cohete es T(plataforma + (x, y, 0)) · Rz(giro). El motor (la llama) sólo se enciende mientras el incremento en Y es positivo, es decir, mientras sube.

• **Creación de keyframes POR TECLADO.** Las flechas izquierda/derecha mueven el cohete en X y las flechas arriba/abajo en Y, 0.5 unidades por pulsación (0.05 si se mantiene presionado Shift izquierdo, para ajustes finos). I y O lo giran sobre Z +15° y −15° (5° con Shift). L guarda la pose actual como un keyframe nuevo y J borra los keyframes de la memoria y regresa el cohete a la plataforma. Cada tecla cuenta una sola vez por pulsación:

se compara su estado con el del cuadro anterior, así que dejarla presionada no repite la acción. Cada movimiento y cada keyframe guardado se imprimen en la consola. Mientras se reproduce la animación no se puede editar la pose.

• **Archivo externo keyframescohete.txt.** G escribe todos los keyframes en keyframescohete.txt con std::ofstream en modo trunc, que borra lo que tenía el archivo (sobreescribe). El archivo tiene dos líneas de comentario (#), el número de keyframes y una línea por keyframe con su índice, x, y y giro. H lo lee con std::ifstream: salta los comentarios, toma el número de keyframes y lee cada línea con std::istringstream; después actualiza FrameIndex y regresa el cohete a la pose del keyframe 0. Además, el archivo se lee solo al iniciar el programa, así que la animación guardada se puede reproducir de inmediato con la barra espaciadora.

• **La animación pedida.** Se crearon 11 keyframes, todos con el teclado. Del 0 al 5 el cohete sube en curva hacia +X y se va inclinando: (0, 0, 0°), (0.5, 2, −15°), (1.5, 4, −30°), (3, 6, −45°), (5, 7.5, −60°) y (7, 8, −90°), donde alcanza las 8 unidades en Y ya horizontal. Del 6 al 9 baja en picada con la nariz cada vez más hacia abajo: (9, 7, −120°), (10.5, 5, −150°), (11.5, 2.5, −165°) y (12, 0.6, −180°), cuando la punta toca el suelo. En el keyframe 10, (14.5, −1.35, −270°), el cohete termina de caer hacia adelante y queda acostado sobre el piso.

Secciones de código que modifiqué

Todo se agregó o modificó en practica10.cpp, en la sección de KEYFRAMES, en el ciclo principal y en inputKeyframes(). Se incluye también el contenido de keyframescohete.txt generado por el programa.

1) Datos del keyframe, guardado con L, interpolación y reproducción

```
//////////////////////////////KEYFRAMES (práctica 10): cohete espacial/////////////////////
//posición del cohete parado en el piso (y = -1): la tobera está 1.9 unidades abajo de su origen
const glm::vec3 posBaseCohete(8.0f, 0.9f, -28.0f);
//variables que se animan: desplazamiento respecto a posBaseCohete y giro sobre Z (grados)
float movCohete_x = 0.0f, movCohete_y = 0.0f, giroCohete = 0.0f;
float fuegoCohete = 0.0f;   //tamaño de la llama del motor (0 = apagado)


#define MAX_FRAMES 100
int i_max_steps = 90;       //pasos de interpolación entre dos keyframes (1.5 s a 60 fps)
int i_curr_steps = 0;
typedef struct _frame
{
    //Variables para GUARDAR Key Frames
    float movCohete_x;        //Variable para PosicionX
    float movCohete_y;        //Variable para PosicionY
    float giroCohete;        //Variable para el giro sobre Z
    float movCohete_xInc;    //Variable para IncrementoX
    float movCohete_yInc;    //Variable para IncrementoY
    float giroCoheteInc;    //Variable para IncrementoGiro
}FRAME;


FRAME KeyFrame[MAX_FRAMES];
int FrameIndex = 0;            //cuántos keyframes hay guardados
bool play = false;
int playIndex = 0;
const char* archivoKeyframes = "keyframescohete.txt";


void saveFrame(void) //tecla L: guarda la pose actual como un keyframe nuevo
{
    if (FrameIndex >= MAX_FRAMES)
    {
        printf("Ya no caben mas keyframes (MAX_FRAMES = %d)\n", MAX_FRAMES);
        return;
    }
    KeyFrame[FrameIndex].movCohete_x = movCohete_x;
    KeyFrame[FrameIndex].movCohete_y = movCohete_y;
    KeyFrame[FrameIndex].giroCohete = giroCohete;
    printf("Keyframe %d guardado POR TECLADO (L): x = %.2f  y = %.2f  giro = %.1f\n",
        FrameIndex, movCohete_x, movCohete_y, giroCohete);
    FrameIndex++;
```

```
}


void resetElements(void) //regresa el cohete a la pose del keyframe 0
{
    movCohete_x = KeyFrame[0].movCohete_x;
    movCohete_y = KeyFrame[0].movCohete_y;
    giroCohete = KeyFrame[0].giroCohete;
}


void interpolation(void)
{
    KeyFrame[playIndex].movCohete_xInc = (KeyFrame[playIndex + 1].movCohete_x - KeyFrame[playIndex].movCohete_x) / i_max_steps;
    KeyFrame[playIndex].movCohete_yInc = (KeyFrame[playIndex + 1].movCohete_y - KeyFrame[playIndex].movCohete_y) / i_max_steps;
    KeyFrame[playIndex].giroCoheteInc = (KeyFrame[playIndex + 1].giroCohete - KeyFrame[playIndex].giroCohete) / i_max_steps;
}


void animate(void)
{
    fuegoCohete = 0.0f;
    if (play)
    {
        if (i_curr_steps >= i_max_steps) //fin de animación entre frames?
        {
            playIndex++;
            if (playIndex > FrameIndex - 2)    //Fin de toda la animación con último frame?
            {
                printf("termino la animacion (%d keyframes)\n", FrameIndex);
                playIndex = 0;
                play = false;
            }
            else //Interpolación del próximo cuadro
            {
                i_curr_steps = 0; //Resetea contador
                interpolation();
            }
        }
        else
        {
            //Dibujar Animación
            movCohete_x += KeyFrame[playIndex].movCohete_xInc;
            movCohete_y += KeyFrame[playIndex].movCohete_yInc;
            giroCohete += KeyFrame[playIndex].giroCoheteInc;
            i_curr_steps++;
            //el motor sólo está encendido mientras el cohete sube
            if (KeyFrame[playIndex].movCohete_yInc > 0.0f)
                fuegoCohete = 0.85f + 0.15f * sinf(i_curr_steps * 0.7f);
        }
    }
}
```

2) Escribir/sobreescribir y leer keyframescohete.txt

```
//Escribe (o sobreescribe) todos los keyframes en keyframescohete.txt. Tecla G.
//Formato: líneas de comentario con #, luego el número de keyframes y una línea "x y giro" por keyframe.
void GuardarKeyframesArchivo(void)
{
    std::ofstream archivo(archivoKeyframes, std::ios::out | std::ios::trunc); //trunc: borra lo que tenía
    if (!archivo.is_open())
    {
        printf("No se pudo abrir %s para escribir\n", archivoKeyframes);
        return;
    }
    archivo << "# Keyframes del cohete (practica 10)\n";
    archivo << "# numero de keyframes, y luego: indice movCohete_x movCohete_y giroCohete\n";
    archivo << FrameIndex << "\n";
    for (int i = 0; i < FrameIndex; i++)
        archivo << i << " " << KeyFrame[i].movCohete_x << " " << KeyFrame[i].movCohete_y << " " << KeyFrame[i].giroCohete << "\
n";
    archivo.close();
    printf("Se escribieron %d keyframes en %s (G)\n", FrameIndex, archivoKeyframes);
}


//Lee los keyframes de keyframescohete.txt. Tecla H (y al iniciar el programa).
bool LeerKeyframesArchivo(void)
{
    std::ifstream archivo(archivoKeyframes);
    if (!archivo.is_open())
    {
        printf("No existe %s: crea los keyframes por teclado y guardalos con G\n", archivoKeyframes);
        return false;
    }
    std::string linea;
    int n = -1, leidos = 0;
    while (std::getline(archivo, linea))
    {
        if (linea.empty() || linea[0] == '#') continue; //comentarios
        std::istringstream datos(linea); //se leen los números de la línea con >> (sin sscanf)
        if (n < 0) { datos >> n; continue; } //primera línea con datos: cuántos keyframes
        if (leidos >= n || leidos >= MAX_FRAMES) break;
        int indice;
        float x, y, g;
        if (datos >> indice >> x >> y >> g)
        {
            KeyFrame[leidos].movCohete_x = x;
            KeyFrame[leidos].movCohete_y = y;
            KeyFrame[leidos].giroCohete = g;
            leidos++;
        }
    }
    FrameIndex = leidos;
```

```
    play = false;
    playIndex = 0;
    if (FrameIndex > 0) resetElements();
    printf("Se leyeron %d keyframes de %s (H)\n", FrameIndex, archivoKeyframes);
    return true;
}
```

3) Matriz del cohete y dibujo en el ciclo principal

```
//Matriz del cohete: posición en la plataforma + desplazamiento animado + giro sobre Z
glm::mat4 MatrizCohete()
{
    glm::mat4 m = glm::translate(glm::mat4(1.0f), posBaseCohete + glm::vec3(movCohete_x, movCohete_y, 0.0f));
    m = glm::rotate(m, giroCohete * toRadians, glm::vec3(0.0f, 0.0f, 1.0f));
    return m;
}
...
        //Práctica 10: cohete animado por keyframes (plataforma + cohete)
        plainTexture.UseTexture();
        Material_opaco.UseMaterial(uniformSpecularIntensity, uniformShininess);
        model = glm::translate(glm::mat4(1.0f), glm::vec3(posBaseCohete.x, -0.75f, posBaseCohete.z));
        model = glm::scale(model, glm::vec3(4.0f, 0.5f, 4.0f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
        color = glm::vec3(0.45f, 0.45f, 0.5f);
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        meshListModel[8]->RenderMeshModel();
        Material_brillante.UseMaterial(uniformSpecularIntensity, uniformShininess);
        DibujarCohete(MatrizCohete(), uniformModel, uniformColor);
```

4) Teclado: crear keyframes, guardar en el archivo, leerlo y reproducir

```
void inputKeyframes(bool* keys)
{
    //Cada tecla cuenta UNA vez por pulsación: se compara con su estado en el frame anterior
    //(si se deja presionada no sigue moviendo ni guardando).
    static bool antes[1024] = { false };
    auto presiono = [&](int tecla) { return keys[tecla] && !antes[tecla]; };
    const bool fino = keys[GLFW_KEY_LEFT_SHIFT];
    const float paso = fino ? 0.05f : 0.5f;      //unidades por pulsación
    const float pasoGiro = fino ? 5.0f : 15.0f;  //grados por pulsación


    //reproducir o detener la animación
    if (presiono(GLFW_KEY_SPACE))
    {
        if (play)
        {
            play = false;
            printf("Animacion detenida\n");
        }
        else if (FrameIndex > 1)
        {
            resetElements();
            playIndex = 0;
            i_curr_steps = 0;
            interpolation(); //First Interpolation
            play = true;
            printf("Reproduciendo %d keyframes\n", FrameIndex);
        }
        else
            printf("Se necesitan al menos 2 keyframes para reproducir\n");
    }


    if (!play) //mientras se reproduce no se puede editar la pose
    {
        bool movio = false;
        if (presiono(GLFW_KEY_RIGHT)) { movCohete_x += paso; movio = true; }
        if (presiono(GLFW_KEY_LEFT))  { movCohete_x -= paso; movio = true; }
        if (presiono(GLFW_KEY_UP))    { movCohete_y += paso; movio = true; }
        if (presiono(GLFW_KEY_DOWN))  { movCohete_y -= paso; movio = true; }
        if (presiono(GLFW_KEY_I))     { giroCohete += pasoGiro; movio = true; }
        if (presiono(GLFW_KEY_O))     { giroCohete -= pasoGiro; movio = true; }
        if (movio)
            printf("  pose: x = %.2f  y = %.2f  giro = %.1f\n", movCohete_x, movCohete_y, giroCohete);


        if (presiono(GLFW_KEY_L)) saveFrame();
        if (presiono(GLFW_KEY_J))
        {
            FrameIndex = 0;
            movCohete_x = movCohete_y = giroCohete = 0.0f;
            printf("Keyframes borrados; el cohete regreso a la plataforma (J)\n");
        }
        if (presiono(GLFW_KEY_G)) GuardarKeyframesArchivo();
        if (presiono(GLFW_KEY_H)) LeerKeyframesArchivo();
    }


    for (int k = 0; k < 1024; k++) antes[k] = keys[k];
}
```

5) Lectura automática del archivo al iniciar e instrucciones en la consola

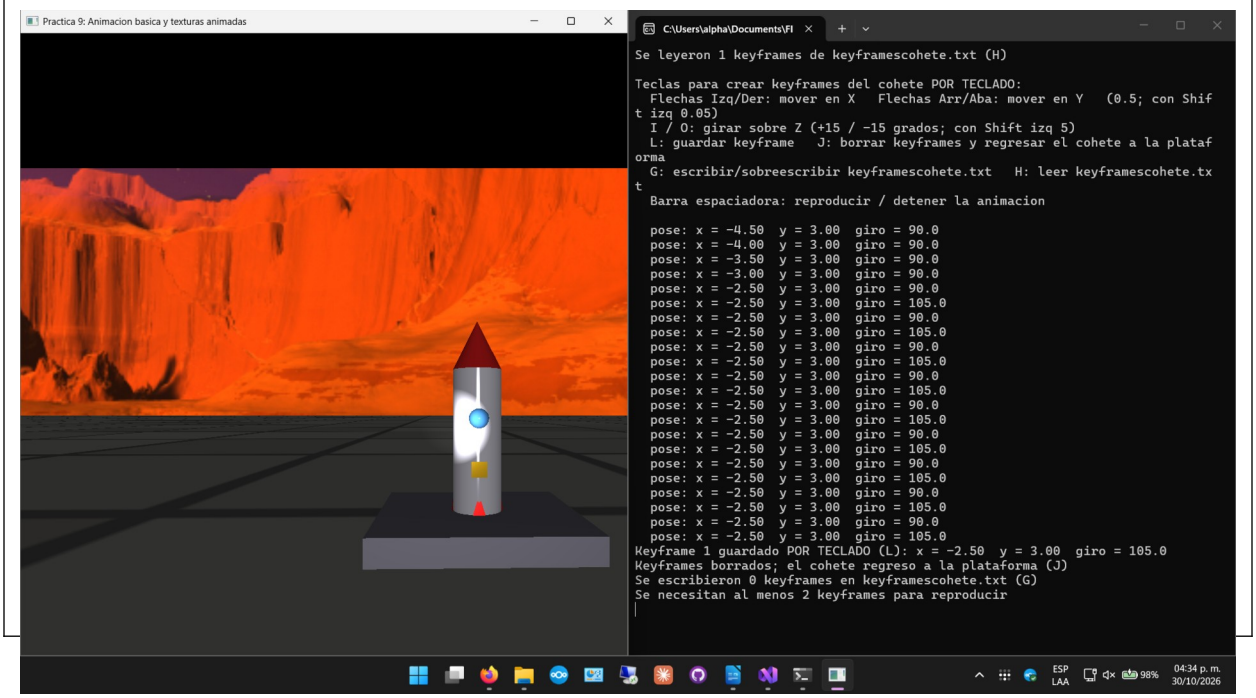
```
    //---------Keyframes del cohete: si ya existe keyframescohete.txt se leen al iniciar
    LeerKeyframesArchivo();
    printf("\nTeclas para crear keyframes del cohete POR TECLADO:\n");
    printf("  Flechas Izq/Der: mover en X   Flechas Arr/Aba: mover en Y   (0.5; con Shift izq 0.05)\n");
    printf("  I / O: girar sobre Z (+15 / -15 grados; con Shift izq 5)\n");
    printf("  L: guardar keyframe   J: borrar keyframes y regresar el cohete a la plataforma\n");
    printf("  G: escribir/sobreescribir keyframescohete.txt   H: leer keyframescohete.txt\n");
    printf("  Barra espaciadora: reproducir / detener la animacion\n\n\n");
```

6) Archivo keyframescohete.txt generado con la tecla G

```
# Keyframes del cohete (practica 10)
# numero de keyframes, y luego: indice movCohete_x movCohete_y giroCohete
11
0 0 0 0
1 0.5 2 -15
2 1.5 4 -30
3 3 6 -45
4 5 7.5 -60
5 7 8 -90
6 9 7 -120
7 10.5 5 -150
8 11.5 2.5 -165
9 12 0.6 -180
10 14.5 -1.35 -270
```

Capturas de pruebas de funcionamiento

Capturas de los keyframes creados por teclado y de la animación en ejecución.



Problemas encontrados durante el desarrollo

1.- El código base animaba un modelo que no estaba en el material: El main de keyframes cargaba Models/uh60.obj (el helicóptero), que no venía en el zip. Se cambió el objeto animado por el cohete de la Práctica 3. Como meshListModel[7] ya era el quad del humo, las primitivas del cohete quedaron en meshListModel[8] a [12] y se ajustaron los índices en DibujarCohete().

2.- Una tecla presionada se repetía en cada cuadro: inputKeyframes() se llama en cada cuadro, así que dejar presionada una flecha movía el cohete muchas veces y L podía guardar varios keyframes iguales. El código base lo evitaba con banderas y una segunda tecla para volver a habilitar (L y P, 1 y 2), lo que hace lento crear muchos keyframes. Se guardó el estado de las teclas del cuadro anterior en un arreglo y cada acción sólo ocurre en el cuadro en que la tecla pasa de suelta a presionada.

3.- Error C4996 al compilar en Visual Studio: La lectura del archivo usaba sscanf, y como el proyecto tiene activada la comprobación SDL, Visual Studio la marca como función insegura y la trata como error (C4996: 'sscanf': This function or variable may be unsafe). Se cambió por std::istringstream, leyendo cada línea con >>. Por la misma razón el archivo se abre con std::ofstream y std::ifstream en lugar de fopen.

4.- Los pasos de 0.5 no alcanzaban para dejar el cohete acostado justo sobre el piso: Acostado, la aleta que queda hacia abajo sobresale 0.55 del eje del cohete, así que su centro debe quedar en y = −1 + 0.55 y el desplazamiento en Y es −1.35, que no es múltiplo de 0.5. Se agregaron pasos finos de 0.05 en X y Y, y de 5° en el giro, con Shift izquierdo. También se usó para que la punta tocara el suelo exactamente en el keyframe 9 (y = 0.6).

5.- Hacia dónde gira el cohete al caer: Después de la picada el cohete toca el suelo con la nariz (giro −180°). Para que caiga hacia adelante, como si siguiera su movimiento, el giro continúa en el mismo sentido hasta −270° (y no regresa a −90°). Como gira apoyado en la punta, que queda 2.5 unidades bajo su origen, el origen avanza 2.5 en X: de 12 a 14.5.

6.- saveFrame() del código base no guardaba el giro: La línea KeyFrame[FrameIndex].giroAvion; no asignaba nada (tenía el comentario "completar"). En la versión del cohete se guardan los tres valores (x, y y giro) y se agregó un tope para no escribir fuera del arreglo de MAX_FRAMES.

Conclusión

En cuanto a la complejidad, la interpolación ya venía resuelta en el código base: sólo hubo que cambiar las variables del avión por las del cohete (x, y y giro). Lo que más trabajo llevó fue el manejo del teclado, para que cada pulsación contara una sola vez y se pudieran hacer ajustes finos, y la lectura y escritura del archivo con un formato sencillo que el programa pudiera volver a leer. Armar la animación pedida requirió pensar cada pose: subir en curva inclinándose, llegar horizontal a las 8 unidades, bajar con la nariz cada vez más abajo y calcular la altura y el giro para que el cohete quedara acostado sobre el piso.

En conclusión, el ejercicio mostró que con la animación por keyframes sólo se definen unas cuantas poses clave y la computadora calcula todos los cuadros intermedios, y que guardar esas poses en un archivo externo permite conservar una animación y reproducirla después sin volver a crearla.

Bibliografía

Angel, E., & Shreiner, D. (2012). Interactive computer graphics: A top-down approach with shader-based OpenGL (6a ed.). Addison-Wesley.

cppreference.com. (s.f.). std::basic_fstream. https://en.cppreference.com/w/cpp/io/basic_fstream

cppreference.com. (s.f.). std::basic_istringstream. https://en.cppreference.com/w/cpp/io/basic_istringstream

GLFW. (s.f.). Input guide. https://www.glfw.org/docs/latest/input_guide.html

Khronos Group. (s.f.). Keyframe animation. OpenGL Wiki. https://www.khronos.org/opengl/wiki/Keyframe_Animation

Microsoft. (s.f.). Compiler warning (level 3) C4996. Microsoft Learn. https://learn.microsoft.com/en-us/cpp/error-messages/compiler-warnings/compiler-warning-level-3-c4996

Vries, J. de. (2020). Learn OpenGL: Transformations. https://learnopengl.com/Getting-started/Transformations